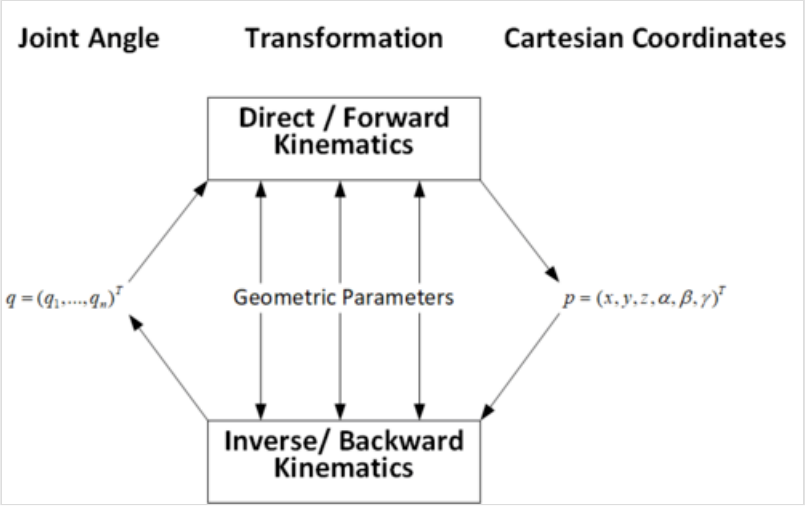
Inverse kinematics

In computer animation and robotics, **inverse kinematics** is the mathematical process of calculating the variable joint parameters needed to place the end of a kinematic chain, such as a robot manipulator or animation character's skeleton, in a given position and orientation relative to the start of the chain. Given joint parameters, the position and orientation of the chain's end, e.g. the hand of the character or robot, can typically be calculated directly using multiple applications of trigonometric formulas, a process known as forward kinematics. However, the reverse operation is, in general, much more challenging.[1][2][3]



Forward vs. inverse kinematics

Inverse kinematics is also used to recover the movements of an object in the world from some other data, such as a film of those movements, or a film of the world as seen by a camera which is itself making those movements. This occurs, for example, where a human actor's filmed movements are to be duplicated by an animated character.

Robotics

In robotics, inverse kinematics makes use of the kinematics equations to determine the joint parameters that provide a desired configuration (position and rotation) for each of the robot's end-effectors.[4] This is important because robot tasks are performed with the end effectors, while control effort applies to the joints. Determining the movement of a robot so that its end-effectors move from an initial configuration to a desired configuration is known as motion planning. Inverse kinematics transforms the motion plan into joint actuator trajectories for the robot.[2] Similar formulas determine the positions of the skeleton of an animated character that is to move in a particular way in a film, or of a vehicle such as a car or boat containing the camera which is shooting a scene of a film. Once a vehicle's motions are known, they can be used to determine the constantly-changing viewpoint for computer-generated imagery of objects in the landscape such as buildings, so that these objects change in perspective while themselves not appearing to move as the vehicle-borne camera goes past them.

The movement of a kinematic chain, whether it is a robot or an animated character, is modeled by the kinematics equations of the chain. These equations define the configuration of the chain in terms of its joint parameters. Forward kinematics uses the joint parameters to compute the configuration of the chain, and inverse kinematics reverses this calculation to determine the joint parameters that achieve a desired configuration.[5][6][7]

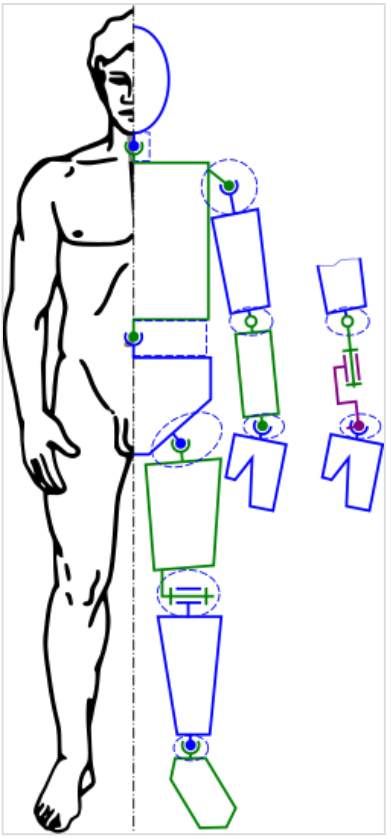
Kinematic analysis

Kinematic analysis is one of the first steps in the design of most industrial robots. Kinematic analysis allows the designer to obtain information on the position of each component within the mechanical system. This information is necessary for subsequent dynamic analysis along with control paths.

Inverse kinematics is an example of the kinematic analysis of a constrained system of rigid bodies, or kinematic chain. The kinematic equations of a robot can be used to define the loop equations of a complex articulated system. These loop equations are non-linear constraints on the configuration parameters of the system. The independent parameters in these equations are known as the degrees of freedom of the system.

While analytical solutions to the inverse kinematics problem exist for a wide range of kinematic chains, computer modeling and animation tools often use Newton's method to solve the non-linear kinematics equations.[2] When trying to find an analytical solution it is often convenient to exploit the geometry of the system and decompose it using subproblems with known solutions.[8][9]

Other applications of inverse kinematic algorithms include interactive manipulation, animation control and collision avoidance.



A model of the human skeleton as a kinematic chain allows positioning using inverse kinematics.

Inverse kinematics and 3D animation

Inverse kinematics is important to game programming and 3D animation, where it is used to connect game characters physically to the world, such as feet landing firmly on top of terrain (see [10] for a comprehensive survey on Inverse Kinematics Techniques in Computer Graphics (http://www.andreasaristidou.com/publications/papers/IK_survey.pdf)).

An animated figure is modeled with a skeleton of rigid segments connected with joints, called a kinematic chain. The kinematics equations of the figure define the relationship between the joint angles of the figure and its pose or configuration. The forward kinematic animation problem uses the kinematics equations to determine the pose given the joint angles. The *inverse kinematics problem* computes the joint angles for a desired pose of the figure.

It is often easier for computer-based designers, artists, and animators to define the spatial configuration of an assembly or figure by moving parts, or arms and legs, rather than directly manipulating joint angles. Therefore, inverse kinematics is used in computer-aided design systems to animate assemblies and by computer-based artists and animators to position figures and characters.

The assembly is modeled as rigid links connected by joints that are defined as mates, or geometric constraints. Movement of one element requires the computation of the joint angles for the other elements to maintain the joint constraints. For example, inverse kinematics allows an artist to move the hand of a 3D human model to a desired position and orientation and have an algorithm select the proper angles of the wrist, elbow, and shoulder joints. Successful implementation of computer animation usually also requires that the figure move within reasonable anthropomorphic limits.

A method of comparing both forward and inverse kinematics for the animation of a character can be defined by the advantages inherent to each. For instance, blocking animation where large motion arcs are used is often more advantageous in forward kinematics. However, more delicate animation and positioning of the target end-effector in relation to other models might be easier using inverted kinematics. Modern digital creation packages (DCC) offer methods to apply both forward and inverse kinematics to models.

Analytical solutions to inverse kinematics

Generic solutions

In some, but not all cases, there exist analytical solutions to inverse kinematic problems. One such example is for a 6-Degrees of Freedom (DoF) robot (for example, 6 revolute joints) moving in 3D space (with 3 position degrees of freedom, and 3 rotational degrees of freedom). If the degrees of freedom of the robot exceeds the degrees of freedom of the end-effector, for example with a 7 DoF robot with 7 revolute joints, then there exist infinitely many solutions to the IK problem, and an analytical solution does not exist. Further extending this example, it is possible to fix one joint and analytically solve for the other joints, but perhaps a better solution is offered by numerical methods (next section), which can instead optimize a solution given additional preferences (costs in an optimization problem).

An analytic solution to an inverse kinematics problem is a closed-form expression that takes the end-effector pose as input and gives joint positions as output, $\boldsymbol{q = f(x)}$. Analytical inverse kinematics solvers can be significantly faster than numerical solvers and provide more than one solution, but only a finite number of solutions, for a given end-effector pose.

Many different programs (Such as FOSS programs IKFast and Inverse Kinematics Library (https://github.com/TheComet/ik)) are able to solve these problems quickly and efficiently using different algorithms such as the FABRIK solver. One issue with these solvers, is that they are known to not necessarily give locally smooth solutions between two adjacent configurations, which can cause instability if iterative solutions to inverse kinematics are required, such as if the IK is solved inside a high-rate control loop.

Ortho-parallel Basis and a Spherical Wrist

Many industrial 6DOF robots feature three rotational joints with intersecting axes ("spherical wrist"). These robots, known as robots with an "Ortho-parallel Basis and a Spherical Wrist," can be defined by 7 kinematic parameters that are distances in their assumed standard geometry.[11] These robots may have up to 8 independent solutions for any given position and rotation of the robot tool head. Open-source solutions for C++[12] and Rust[13] exist. OPW has also been integrated into ROS framework. [14]

Numerical solutions to IK problems

There are many methods of modelling and solving inverse kinematics problems. The most flexible of these methods typically rely on iterative optimization to seek out an approximate solution, due to the difficulty of inverting the forward kinematics equation and the possibility of an empty solution space. The core idea behind several of these methods is to model the forward kinematics equation using a Taylor series expansion, which can be simpler to invert and solve than the original system.

Spherical wrist (axes of the last three joints of the robot intersect).

The Jacobian inverse technique

The Jacobian inverse technique is a simple yet effective way of implementing inverse kinematics. Let there be m variables that govern the forward-kinematics equation, i.e. the position function. These variables may be joint angles, lengths, or other arbitrary real values. If, for example, the IK system lives in a 3-dimensional space, the position function can be viewed as a mapping $p(x) : \mathbb{R}^m \to \mathbb{R}^3$. Let $p_0 = p(x_0)$ give the initial position of the system, and

$$p_1 = p(x_0 + \Delta x)$$

be the goal position of the system. The Jacobian inverse technique iteratively computes an estimate of Δx that minimizes the error given by $||p(x_0 + \Delta x_{\text{estimate}}) - p_1||$.

For small Δx-vectors, the series expansion of the position function gives

$$p(x_1) \approx p(x_0) + J_p(x_0)\Delta x,$$

where $J_p(x_0)$ is the (3 × m) Jacobian matrix of the position function at x_0.

The (i, k)-th entry of the Jacobian matrix can be approximated numerically

$$\frac{\partial p_i}{\partial x_k} \approx \frac{p_i(x_{0,k} + h) - p_i(x_0)}{h},$$

where $p_i(x)$ gives the i-th component of the position function, $x_{0,k} + h$ is simply x_0 with a small delta added to its k-th component, and h is a reasonably small positive value.

Taking the Moore–Penrose pseudoinverse of the Jacobian (computable using a singular value decomposition) and re-arranging terms results in

$$\Delta x \approx J_p^+(x_0)\Delta p,$$

where $\Delta p = p(x_0 + \Delta x) - p(x_0)$.

Applying the inverse Jacobian method once will result in a very rough estimate of the desired Δx-vector. A line search should be used to scale this Δx to an acceptable value. The estimate for Δx can be improved via the following algorithm (known as the Newton–Raphson method):

$$\Delta x_{k+1} = J_p^+(x_k)\Delta p_k$$

Once some Δx-vector has caused the error to drop close to zero, the algorithm should terminate. Existing methods based on the Hessian matrix of the system have been reported to converge to desired Δx values using fewer iterations, though, in some cases more computational resources.

Heuristic methods

The inverse kinematics problem can also be approximated using heuristic methods. These methods perform simple, iterative operations to gradually lead to an approximation of the solution. The heuristic algorithms have low computational cost (return the final pose very quickly), and usually support joint constraints. The most popular heuristic algorithms are cyclic coordinate descent (CCD)[15] and forward and backward reaching inverse kinematics (https://doi.org/10.1016/j.gmod.2011.05.003) (FABRIK).[16]

See also

Physics portal

- 321 kinematic structure
- Arm solution
- Forward kinematic animation
- Forward kinematics
- Jacobian matrix and determinant
- Joint constraints
- Kinematic synthesis
- Kinemation
- Levenberg–Marquardt algorithm
- Motion capture
- Physics engine
- Pseudoinverse
- Ragdoll physics
- Robot kinematics
- Denavit–Hartenberg parameters

References

1. Donald L. Pieper, The kinematics of manipulators under computer control (https://web.archive.org/web/20160924184009/http://www.dtic.mil/cgi-bin/GetTRDoc?AD=AD0680036). PhD thesis, Stanford University, Department of Mechanical Engineering, October 24, 1968.
2. Lynch, Kevin M.; Park, Frank C. (2017-05-25). *Modern Robotics* (https://books.google.com/books?id=5NzFDgAAQBAJ). Cambridge University Press. ISBN 978-1-107-15630-2.

3. Siciliano, Bruno; Khatib, Oussama (2016-06-27). *Springer Handbook of Robotics* (https://books.google.com/books?id=QbgKkAEACAAJ). Springer International Publishing. ISBN 978-3-319-32550-7.
4. Paul, Richard (1981). *Robot manipulators: mathematics, programming, and control : the computer control of robot manipulators* (https://books.google.com/books?id=UzZ3LAYqvRkC). MIT Press, Cambridge, MA. ISBN 978-0-262-16082-7.
5. J. M. McCarthy, 1990, *Introduction to Theoretical Kinematics,* MIT Press, Cambridge, MA.
6. J. J. Uicker, G. R. Pennock, and J. E. Shigley, 2003, **Theory of Machines and Mechanisms,** Oxford University Press, New York.
7. J. M. McCarthy and G. S. Soh, 2010, *Geometric Design of Linkages,* (https://books.google.com/books?id=jv9mQyjRIw4C&dq=geometric+design+of+linkages&pg=PA231) Springer, New York.
8. Paden, Bradley Evan (1985-01-01). *Kinematics and Control of Robot Manipulators* (https://ui.adsabs.harvard.edu/abs/1985PhDT.....94P) (Thesis). Bibcode:1985PhDT.....94P (https://ui.adsabs.harvard.edu/abs/1985PhDT.....94P).
9. Murray, Richard M.; Li, Zexiang; Sastry, S. Shankar; Sastry, S. Shankara (1994-03-22). *A Mathematical Introduction to Robotic Manipulation* (https://books.google.com/books?id=D_PqGKRo7oIC&q=murray+li+sastry). CRC Press. ISBN 978-0-8493-7981-9.
10. A. Aristidou, J. Lasenby, Y. Chrysanthou, A. Shamir. Inverse Kinematics Techniques in Computer Graphics: A Survey (https://doi.org/10.1111/cgf.13310). Computer Graphics Forum, 37(6): 35-58, 2018.
11. Mathias Brandstötter, Arthur Angerer, Michael Hofbaur (2014). An Analytical Solution of the Inverse Kinematics Problem of Industrial Serial Manipulators with an Ortho-parallel Basis and a Spherical Wrist. Proceedings of the Austrian Robotics Workshop 2014. 22-23 May, 2014, Linz, Austria.[1] (https://www.researchgate.net/profile/Mathias-Brandstoetter/publication/264212870_An_Analytical_Solution_of_the_Inverse_Kinematics_Problem_of_Industrial_Serial_Manipulators_with_an_Ortho-parallel_Basis_and_a_Spherical_Wrist/links/53d2417e0cf2a7fbb2e98b09/An-Analytical-Solution-of-the-Inverse-Kinematics-Problem-of-Industrial-Serial-Manipulators-with-an-Ortho-parallel-Basis-and-a-Spherical-Wrist.pdf)
12. opw_kinematics [2] (https://github.com/Jmeyer1292/opw_kinematics)
13. Rust is for Robotics (curated collection), [3] (https://github.com/robotics-rs/robotics.rs)
14. moveit_opw_kinematics_plugin (ROS Wiki) [4] (https://wiki.ros.org/moveit_opw_kinematics_plugin)
15. D. G. Luenberger. 1989. Linear and Nonlinear Programming. Addison Wesley.
16. A. Aristidou, and J. Lasenby. 2011. FABRIK: A fast, iterative solver for the inverse kinematics problem (https://doi.org/10.1016/j.gmod.2011.05.003). Graph. Models 73, 5, 243–260.

External links

- Forward And Backward Reaching Inverse Kinematics (FABRIK) (http://andreasaristidou.com/FABRIK.html)
- Robotics and 3D Animation in FreeBasic (https://sites.google.com/site/proyectosroboticos/cinematica-inversa-iii) (in Spanish)
- Analytical Inverse Kinematics Solver (http://openrave.org/docs/latest_stable/) - Given an OpenRAVE robot kinematics description, generates a C++ file that analytically solves for the complete IK.
- Inverse Kinematics algorithms (https://web.archive.org/web/20110823191421/http://freespace.virgin.net/hugo.elias/models/m_ik2.htm)
- Robot Inverse solution for a common robot geometry (http://www.learnaboutrobots.com/inverseKinematics.htm)

- HowStuffWorks.com article *How do the characters in video games move so fluidly?* (http://entertainment.howstuffworks.com/question538.htm) with an explanation of inverse kinematics
- 3D animations of the calculation of the geometric inverse kinematics of an industrial robot (https://www.youtube.com/watch?v=3s2x4QsD3uM)
- 3D Theory Kinematics (http://www.euclideanspace.com/physics/kinematics/joints/index.htm)
- Protein Inverse Kinematics (http://cnx.org/content/m11613/latest/)
- Simple Inverse Kinematics example with source code using Jacobian (https://web.archive.org/web/20110811105301/http://sites.google.com/site/diegopark2/computergraphics)
- Detailed description of Jacobian and CCD solutions for inverse kinematics (http://billbaxter.com/courses/290/html/index.htm)
- Autodesk HumanIK (http://www.autodesk.com/products/humanik)
- A 3D visualization of an analytical solution of an industrial robot (https://www.youtube.com/watch?v=3s2x4QsD3uM)
